Monitoramento da Qualidade do Ar

Universidade Comunitária da Região de Chapecó - Unochapecó

Disciplina: Bancos de Dados e Novas Tecnologias - Prof. Monica Tissiani De Toni Pereira

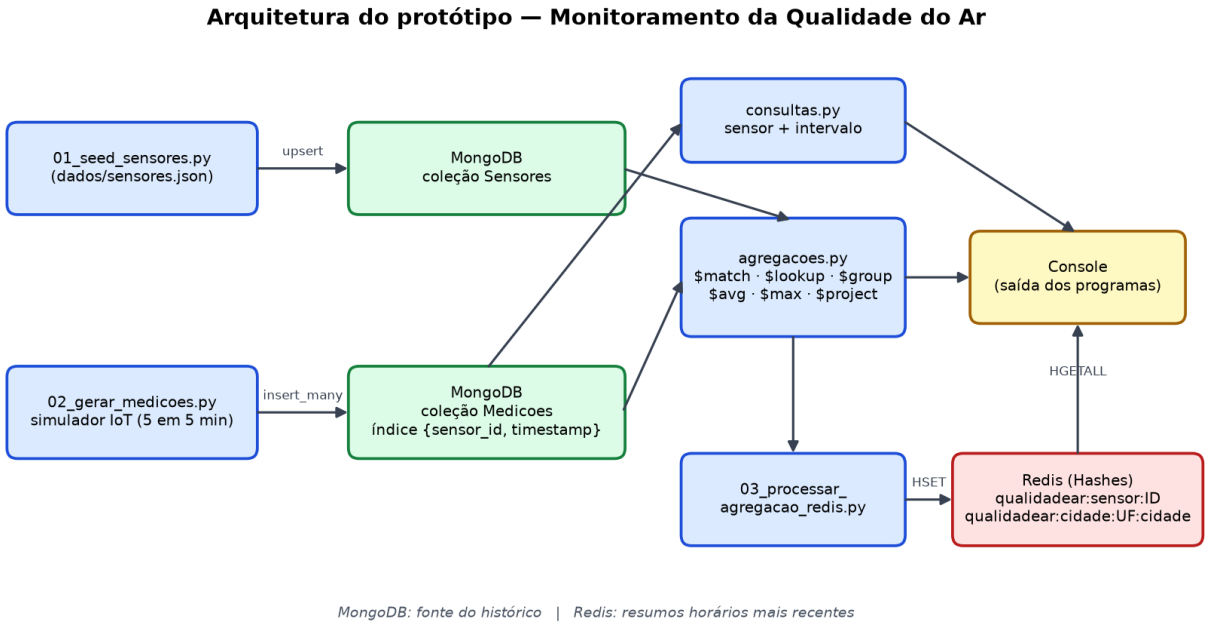
Integrantes: Ana Clara Jesus, Thais Zanella

1. Introdução

Uma empresa de monitoramento ambiental usa dispositivos IoT para medir PM2.5, PM10, CO_2, temperatura e umidade em diferentes localidades. Este trabalho implementa um protótipo que armazena o histórico das medições, calcula resumos horários e os disponibiliza para consulta rápida, sem interface gráfica: toda a saída é apresentada no console.

O MongoDB guarda os sensores e as medições históricas. O Redis guarda, em Hashes, os resumos horários mais recentes por sensor e por cidade. Os programas são escritos em Python.

2. Arquitetura



Fluxo: “01_seed_sensores.py” cadastra os sensores, “02_gerar_medicoes.py” simula as leituras, “agregacoes.py” calcula os resumos com o pipeline de agregação do MongoDB, “03_processar_agregacao_redis.py” grava os resultados em Hashes do Redis, “consultas.py” consulta medições por sensor e intervalo. O MongoDB e o Redis rodam em contêineres Docker (“docker-compose.yml”).

2.1 Etapas do pipeline

O pipeline analítico foi concebido em três etapas complementares, que simulam uma infraestrutura de monitoramento ambiental em tempo real, com analítica avançada e leitura de alto desempenho. Cada etapa consome o resultado da anterior.

Etapa / Script	Responsabilidade	Tecnologia / Coleção	Integração com o fluxo
Etapa 1 01_seed_sensores.py	Cadastro e persistência dos metadados dos sensores ambientais.	MongoDB (coleção Sensores)	Cria os registros dos sensores (ID, localização, cidade, UF) necessários para o cruzamento espacial.
Etapa 2 02_gerar_medicoes.py	Simulação de leituras temporais de poluentes (PM2.5, PM10, CO_2, etc.).	MongoDB (coleção Medicoes)	Gera a massa de dados associada aos IDs dos sensores no intervalo de tempo especificado.
Etapa 3 03_processar_agregacao_redis.py	Agregação analítica e armazenamento em cache de baixa latência.	MongoDB Aggregation e Redis Hashes	Consome os dados das Etapas 1 e 2, processa as métricas calculadas e otimiza a camada de consulta.

3. Modelagem

3.1 MongoDB (banco “QualidadeAr”)

Sensores - um documento por sensor, com “_id” (ex.: “S001”), “modelo”, “descricao”, “localizacao” (“cidade”, “uf”, “latitude”, “longitude”), “ativo” e “grandezas”. Foram cadastrados 6 sensores em 3 cidades (Chapecó, Florianópolis e Itajaí), dois por cidade.

Medições - um documento por leitura: “sensor_id”, “timestamp”, “pm25”, “pm10”, “co2”, “temperatura”, “umidade”. Há 24h de dados por sensor, a cada 5 minutos: 288 por sensor, 1.728 no total.

Medições inválidas: O simulador injeta uma pequena proporção de leituras com falha (padrão 1,5%, valores negativos). Todas as agregações usam apenas medições válidas (“qt_medicoes” conta somente as válidas).

3.2 Redis

Hash	Padrão de chave	Exemplo
Por sensor	qualidadear:sensor:<ID>	qualidadear:sensor:S001
Por cidade	qualidadear:cidade:<UF>:<cidade>	qualidadear:cidade:SC:chapeco

Campos: “inicio_periodo”, “fim_periodo”, “qt_medicoes”, “pm25_media”, “pm10_media”, “co2_media”, “temperatura_media”, “umidade_media”, “pm25_max”, “pm10_max”.

4. Pipeline de agregação e cache (Etapa 3)

O script “03_processar_agregacao_redis.py” é o núcleo analítico do protótipo e integra as Etapas 1 e 2. Seu funcionamento segue quatro passos:

1. **Leitura:** consulta, no MongoDB, os sensores cadastrados (Etapa 1) e as medições geradas (Etapa 2), usando o ID do sensor para o cruzamento com a localização (cidade e UF).

2. **Agregação:** executa, por meio das funções de “agregacoes.py”, o pipeline de agregação do MongoDB (“$match”, “$lookup”, “$group”, “$avg”, “$max”, “$project”) sobre a janela de tempo [início, fim), considerando apenas medições válidas.

3. **Cálculo das métricas:** médias de PM2.5, PM10, CO_2, temperatura e umidade, máximos de PM2.5 e PM10 e a quantidade de medições válidas, por sensor e por cidade.

4. **Cache:** grava os resultados em Hashes do Redis (chaves descritas na seção 3.2), de modo que a consulta dos resumos mais recentes não precise repetir a agregação no MongoDB e tenha resposta em tempo sub-milissegundo.

Dessa forma, o MongoDB permanece como fonte de verdade do histórico, enquanto o Redis atua como camada de leitura otimizada para os resumos horários.

5. Implementação

Arquivo	**Função**
conexao.py	Conexões com MongoDB e Redis a partir do “.env”
01_seed_sensores.py	Cadastra os sensores de “dados/sensores.json” (upsert, idempotente)
02_gerar_medicoes.py	Gera as medições simuladas (parâmetros “--inicio”, “--horas”, “--seed”, “--taxa-falhas”); remove o período antes de inserir, para não duplicar
consultas.py	Subcomandos “sensores”, “ultima”, “sensor”, “falhas” e “plano”
agregacoes.py	Resumos por sensor, por cidade, horários e comparação de médias
03_processar_agregacao_redis.py	Executa as agregações e grava as Hashes no Redis (ver seção 4)

6. Ferramentas utilizadas

Ferramenta	Uso
Python	Linguagem dos programas
MongoDB	Armazenamento do histórico e pipelines de agregação
Redis	Hashes com os resumos recentes
Docker Compose	Subir MongoDB e Redis
GitHub	Controle de versão e entrega
PyMongo 4.10.1, redis-py 5.2.1, python-dotenv 1.0.1, Unidecode 1.3.8, tabulate 0.9.0	Bibliotecas Python
Claude Code (Anthropic), modelo Claude Sonnet 5.5	Ferramenta de IA usada como apoio na escrita e revisão de código, nos testes, no diagrama e na redação deste relatório.

7. Dificuldades encontradas e soluções

Dificuldade: Média por cidade com sensores de quantidades diferentes de registros

Solução: “$group” direto sobre as medições da cidade (média global), com o comando “comparar” para mostrar a diferença em relação à média das médias

Dificuldade: Datas sem fuso gerando janelas deslocadas

Solução: Cliente MongoDB com “tz_aware=True” e todas as datas em UTC; janela [início, fim)

Dificuldade: Medições com falha distorcendo médias e máximos

Solução: Filtro de medições válidas no “$match” de todas as agregações

Dificuldade: Duplicação de dados ao rodar o gerador mais de uma vez

Solução: “02_gerar_medicoes.py” remove o período antes de inserir; “01_seed_sensores.py” usa upsert

8. Passo a passo para reproduzir

1 - Clonar o repositório e abrir a pasta do projeto

2 - Criar ambiente virtual e instalar as dependências

macOS ou Linux:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Windows (PowerShell):

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

```
pip install -r requirements.txt
```

3 - Criar o .env (duplicar o .env.example)

4 - Subir o MongoDB e o Redis

```
docker compose up -d
docker ps
```

5 - Executar os programas (dentro da pasta codigo), na ordem das etapas do pipeline

```
cd codigo
python 01_seed_sensores.py              # Etapa 1: cadastra os 6 sensores
python 02_gerar_medicoes.py            # Etapa 2: gera as 1.728 medições
python 03_processar_agregacao_redis.py # Etapa 3: calcula os resumos e grava
                                       # no Redis
```

6 - Consultas e agregações

```
python consultas.py sensores
python consultas.py sensor S001 --inicio 2026-10-20T10:00:00Z \
    --fim 2026-10-20T10:30:00Z
python consultas.py plano S001 --inicio 2026-10-20T10:00:00Z \
    --fim 2026-10-20T11:00:00Z
python agregacoes.py sensor --inicio 2026-10-20T10:00:00Z
python agregacoes.py cidade --inicio 2026-10-20T10:00:00Z
python agregacoes.py comparar --inicio 2026-10-20T10:00:00Z
```